Bilan et échantillons

1. Détail de la réalisation

Au niveau des tables, il s'agit de réaliser sur la table visite une modification (champs bilan, premierMedicament et secondMedicament) et des ajouts dans la table Echantillon.

2. Le trigger avantAjoutVisite

Le déclencheur avantMajVisite a déjà été mis en place lors de la réalisation de l'ajout d'un rendez-vous mais ce dernier doit peut-être évoluer si certains contrôles n'ont pas été pris en compte :

Rappel de l'ensemble des règles :

La visite doit appartenir au visiteur connecté

Champs strictement non modifiables

id : l'identifiant de la visite ne peut jamais changer.
idVisiteur : le visiteur assigné à la visite ne peut pas être modifié.
idPraticien : le praticien assigné à la visite ne peut pas être modifié.
idMotif : le motif de la visite ne peut pas être changé.

Verrouillage d'une visite clôturée

Si la visite est déjà clôturée (bilan renseigné), aucune modification n'est autorisée. Cela empêche de modifier la date, le bilan, les médicaments ou toute autre information.

Interdiction de modification simultanée

On ne peut jamais modifier la date de la visite (dateEtHeure) et les informations médicales (bilan, premierMedicament, secondMedicament) en même temps.
Soit on reprogramme la visite → seule la date peut changer.
Soit on complète la visite → seul le bilan et les médicaments peuvent changer.

Contrôles lors de la reprogrammation (dateEtHeure)

La date doit être au moins 1 heure dans le futur.
La date ne peut pas être à plus de 2 mois de la date actuelle.
La date ne doit pas tomber un dimanche.
La visite doit être programmée entre 8h00 et 19h00 inclus.
Il doit y avoir au moins 2 heures d'écart entre deux visites pour le même visiteur.

Contrôles lors de la saisie du bilan et des médicaments

La visite ne peut être complétée qu'après sa réalisation (date passée).
Le bilan est obligatoire si on complète la visite.
Le premier médicament est obligatoire et il doit exister.
Le second médicament est facultatif, mais s'il est renseigné, il doit être différent du premier médicament et exister.

3. Le trigger avantAjoutEchantillon

Ensemble des règles à respecter

L'ajout d'échantillon n'est possible que pour une visite dont le bilan est renseigné
Le médicament est obligatoire et il doit exister.

L'id de la visite est obligatoire la visite dont exister et faire partie des visites du visiteur (ou des visites si l'ajout est réalisé par 'root')

La quantité est obligatoire et doit être comprise entre 1 et 10
Le nombre de médicaments distribués ne doit pas dépasser 10

4. Les procédures stockées

La mise à jour passera par l'utilisation de procédures stockées :

enregistrerBilanVisite(_idVisite, _bilan, _premierMedicament, _secondMedicament) pour mettre à jour la table visite

ajouterEchantillon(_idVisite, _idMedicament, _quantite) pour ajouter ou modifier les échantillons

Ces procédures prendront en charge au minimum la vérification des valeurs transmises afin d'éviter l'exécution de la requête update ou insert

Vérification du paramètre _idVisite : (Null, '', non numérique)
Vérification du paramètre _bilan qui doit être renseigné
Vérification du paramètre _premierMedicament qui doit être renseigné
Vérification du second médicament : s'il est renseigné il doit être différent du premier

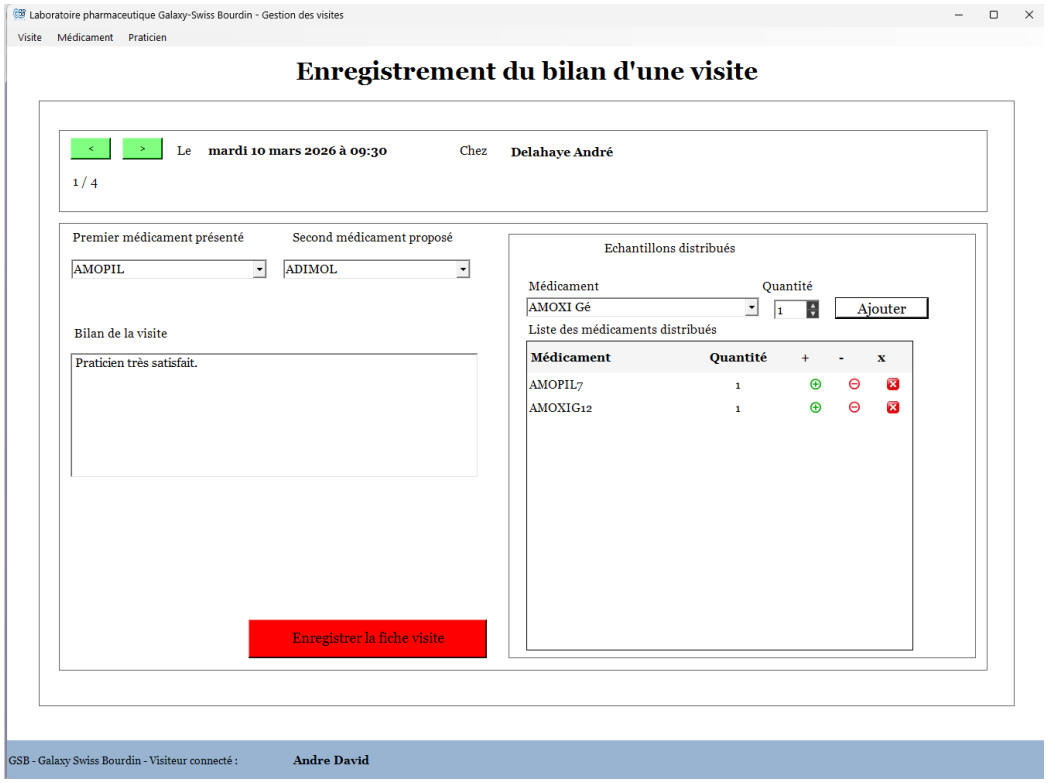
Vérification du paramètre _idVisite : (Null, '', non numérique)
Vérification du paramètre _idMedicament qui doit être renseigné

5. La méthode Passerelle.enregistrerBilanVisite

La procédure enregistrerBilan doit permettre d'enregistrer la visite (modification d'un enregistrement dans la table visite) en y intégrant les médicaments distribués (ajout de 0 à n enregistrement dans la table medicamentDistribué)
Pour s'assurer de l'unicité de l'opération toutes les actions réalisées sont placées dans une transaction.

```
// Déclaration d'un transaction
MySqlTransaction uneTransaction = cnx.BeginTransaction();
// attacher la commande à la transcation
 cmd.Transaction = uneTransaction;
// annuler la transaction
 uneTransaction.Rollback();
  uneTransaction.Commit();
```

6. Proposition de l'interface graphique



Principaux composants

4 panels : panelCentral, panetHaut, panelDroit, panelGauche (pas de dock)
Un textBox txtBilan multiligne
3 labels en rouge pour les messages d'erreur **msgPremierMedicament**, **msgSecondMedicament**, et **msgBilan**

Pour la saisie des échantillons de médicament distribué :
Un ComboBox **cbxEchantillon** pour sélectionner le médicament
Un NumericUpDown pour saisie la quantité **cptQuantite** (minimun 1 ; maximun 10 et value 1)
Un bouton btnAjouter pour ajouter l'échantillon dans le dataGridView **dgvEchantillon**

Pour l'affichage et la suppression des échantillons
Un dataGridView dgvEchantillon, composé de 4 colonnes
- Colonne 0 : cachée de type TextBox contenant l'objet médicament
- Colonne 1 : de type TextBox contenant le nom du médicament
- Colonne 2 : de type TextBox contenant la quantité
- Colonne 3 à 5 : de type Image

Indication

Aucun composant ne doit s'agrandir ici. Pour améliorer le rendu quelque soit la dimension de l'écran, le panel englobant tous les contrôles, ne doit être attaché à aucun bord et la procédure centrerFormulaire exécutée au chargement et lors du redimensionnement du formulaire va assurer le centrage horizontal et vertical

```
private void centrerFormulaire() {
    panelCentral.Left = (this.ClientSize.Width - panelCentral.Width) / 2;
    panelCentral.Top = (this.ClientSize.Height - panelCentral.Height) / 2;
 }
```

7. La gestion des visites non complétées avec les boutons 'Suivant' et 'Précédent'

7.1. Principe général

L'objectif est de permettre à l'utilisateur de parcourir une liste de visites non complétées :

avec Suivant → aller à la visite suivante
avec Précédent → revenir à la visite précédente
avec un affichage du type "position / total" (ex : 2 / 5)

Pour cela nous avons besoin en termes de donnée :

d'une collection d'objets : **private List<Visite> lesVisitesNonCompletees = new();** contenant uniquement les visites déjà passées (DateEtHeure <= Now) et sans bilan (Bilan == null)

de l'index de la visite courante : **private int indexVisiteCourante = 0;**

7.2. Chargement initial des visites

```
private void chargerLesVisites() {
   lesVisitesNonCompletees = session!.MesVisites
      .FindAll(x => x.DateEtHeure <= DateTime.Now && x.Bilan == null);

   if (lesVisitesNonCompletees.Count == 0)   {
      message.Text = "Toutes vos fiches sont complétées";
      panelCentral.Visible = false;
      return;
   }

   panelCentral.Visible = true;
   indexVisiteCourante = 0;
   afficherVisite();
   MettreAJourNavigation();
}
```

7.3. Bouton "Suivant" et "Précédent"

Ils vont appeler la méthode changerVisite qui va se positionner sur la bonne visite

```
private void btnPrecedent_Click(object sender, EventArgs e) {
   changerVisite(-1);
}

private void btnSuivant_Click(object sender, EventArgs e) {
   changerVisite(1);
}
```

```
private void changerVisite(int deplacement) {
   int count = lesVisitesNonCompletees.Count;

   if (count == 0)     {
      return;
   }

   // Calcul de l'index de la visite courante après déplacement en utilisant l'opérateur modulo pour
assurer un déplacement circulaire
   indexVisiteCourante = (indexVisiteCourante + deplacement + count) % count;

   afficherVisite();
   mettreAJourNavigation();
}
```

En utilisant l'opérateur modulo % on assure une rotation circulaire et en ajoutant count on s'assure d'avoir toujours une valeur positive lorsque indexVisiteCourante = 0 et deplacement = -1

Dans ce cas avec count=10 on se retrouve bien sur la visite 9 (la dernière qui précède bien la première)

La méthode mettreAjourNavigation doit activer/désactiver les boutons et peut utiliser un label 'message' pour afficher un indicateur de positionnement

```
private void mettreAJourNavigation() {
   int count = lesVisitesNonCompletees.Count;

   btnSuivant.Enabled = count > 1;
   btnPrecedent.Enabled = count > 1;

   message.Visible = count > 0;

   if (count > 0)   {
      message.Text = $"{indexVisiteCourante + 1} / {count}";
   } else   {
      message.Text = "Toutes vos fiches sont complétées";
   }
}
```

Dans chaque méthode ayant besoin de récupérer la visite affichée :

```
Visite visite = lesVisitesNonCompletees[indexVisiteCourante];
```

8. Enregistrement du bilan et des échantillons

Il faut ici commencer par mettre à jour les objets métier avant de mettre à jour la base de données car la méthode Passerelle.enregistrerBilan attend en paramètre l'objet visite qui doit contenir les échantillons remis au praticien.

Un fois la visite mise à jour il faut la retirer des visites non complétées et par conséquent afficher la visite suivante ou précédente s'il en reste encore une.

La solution consiste à appeler la méthode **changerVisite en lui passant 0 en paramètre** afin de demander d'afficher la visite actuellement

```
private void enregistrer()  {
    bool premierMedicamentOk = controlerPremierMedicament();
    bool secondMedicamentOk = controlerSecondMedicament();
    bool bilanOk = controlerBilan();
    if (premierMedicamentOk && secondMedicamentOk && bilanOk)   {
        Medicament premierMedicament = (Medicament)cbxPremierMedicament.SelectedItem!;
        Medicament? secondMedicament = (cbxSecondMedicament.SelectedIndex >= 0) ?
(Medicament)cbxSecondMedicament.SelectedItem! : null;
        Visite visite = lesVisitesNonCompletees[indexVisiteCourante];
        try  {
            // mise à jour de l'objet Visite en mémoire
            visite.enregistrerBilan(txtBilan.Text, premierMedicament, secondMedicament);

            // Enregistrement base de données
            Passerelle.enregistrerBilan(visite);

            MessageBox.Show("Votre fiche visite est maintenant archivée", "Bilan enregistré",
MessageBoxButtons.OK, MessageBoxIcon.Information);

            // Supprimer la visite de la liste
            lesVisitesNonCompletees.RemoveAt(indexVisiteCourante);

            // Afficher la prochaine visite (ou revenir au début si liste vide)
            if (lesVisitesNonCompletees.Count > 0)            {
                // on peut appeler changerVisite(0) pour réafficher la visite courante sans changer d'index
                changerVisite(0);
            } else  {
                message.Text = "Toutes vos fiches sont complétées";
                panelCentral.Visible = false;
                mettreAJourNavigation();
            }
        }
        catch (Exception ex)   {
            MessageBox.Show("Une erreur est survenue lors de l'enregistrement du bilan : " + ex.Message,
"Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```